

Resampling Linear Models

Bootstrapping Linear Models

Consider the regression model

$$y_i = \beta x_i + \epsilon_i$$

We wish to test whether the regression is significant. We usually do this based on the assumption that the residuals have a normal distribution. However, there are situations where we are not able to make this assumption. The bootstrap gives us a means to test for significance of the regression without assuming normality. There are two ways to do the bootstrap with a linear model. We can either bootstrap the residuals or we can bootstrap the pairs:

Bootstrap the residuals

In order to assess significance we need the distribution for the residuals. We can approximate this with the empirical distribution. We estimate the residuals as

$$\hat{\epsilon} = y_i - \hat{\beta}x_i$$

We then generate bootstrap values for the response variable as

$$y_i^* = \hat{\beta}x_i + \epsilon_i^*$$

Where ϵ_i^* is sampled with replacement from the fitted residuals $\hat{\epsilon}_i$. That is, each bootstrap y value consists of the nonrandom fitted part $\hat{\beta}x_i$ and the random bootstrapped part ϵ_i^* .

Then, the bootstrap least squares estimate $\hat{\beta}^*$ is the least squares estimate from the bootstrap data set.

Bootstrap pairs

The second method for bootstrapping linear models is to bootstrap the pairs. The original data is

$$\begin{array}{cc} x_1 & y_1 \\ x_2 & y_2 \\ . & . \\ . & . \\ x_n & y_n \end{array}$$

We get a bootstrap data set by sampling with replacement from the pairs:

$$x_{i_1} \quad y_{i_1}$$

$$\begin{array}{cc} x_{i_2} & y_{i_2} \\ & \vdots \\ & \vdots \\ x_{i_n} & y_{i_n} \end{array}$$

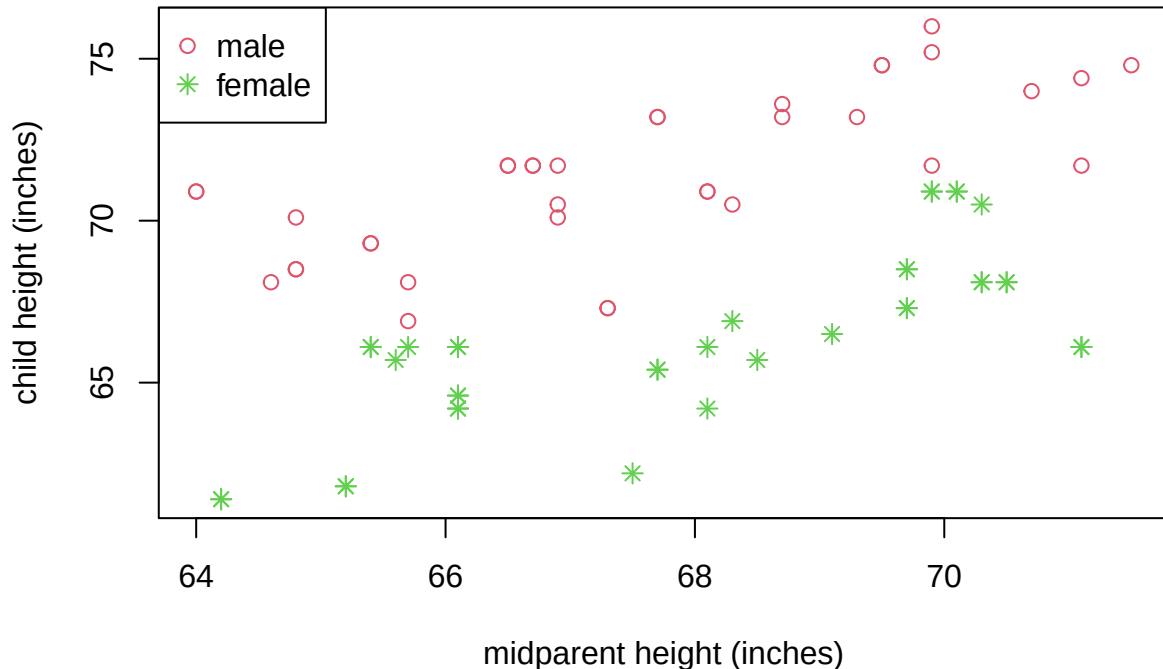
Where $i_1 \dots i_n$ is a random sample with replacement from the integers 1 to n . We calculate $\hat{\beta}^*$ from each bootstrap data set and use the set of $\hat{\beta}^*$ values to approximate the distribution of $\hat{\beta}$. Bootstrapping the pairs is less sensitive to the assumptions and does not depend on the model actually being right. When we bootstrap the residuals then we are using the model to calculate the residuals. If the model is not correct, then the residuals will not be correct and the empirical distribution for the residuals will not be correct. If the model *is* correct, then bootstrapping the residuals will give better error estimates.

Example: Swiss Height Data

Now, will do an example with a data set that consists of the heights of college students in Switzerland, along with the heights of their parents. The goal is to see how the student's heights depend on their parent's heights. The last column is the "mid-parent height" - that is, the average of the two parent's heights. We will do regression of the child's height on the midparent height. From the plot below, we can clearly see that males are different (taller) than females. Thus, we will focus the analysis on females.

##	Child	Mother	Father	Gender	Midparent
## 1	68.1	64.6	64.6	M	64.6
## 2	73.6	66.1	71.3	M	68.7
## 3	74.8	70.1	72.8	M	71.5
## 4	66.1	63.8	67.7	F	65.7
## 5	73.2	69.3	69.3	M	69.3
## 6	76.0	68.9	70.9	M	69.9

Swiss Height Data



Example with bootstrapping residuals:

```
library(boot)

#reduce to female students only:
datF<-dat[Gender=="F",]
detach(dat)
attach(datF)

fit <- lm(Child~Midparent) #fit linear model
summary(fit)

##
## Call:
## lm(formula = Child ~ Midparent)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.6441 -0.8340 -0.3404  1.5596  2.8210
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.9869     9.2471   0.323   0.749
## Midparent     0.9312     0.1360   6.850 5.98e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.762 on 35 degrees of freedom
## Multiple R-squared:  0.5727, Adjusted R-squared:  0.5605
## F-statistic: 46.92 on 1 and 35 DF,  p-value: 5.979e-08
```

Make data frame with the residuals and the fitted model added in:

```
datF1 <- data.frame(datF, res = resid(fit), fitted = fitted(fit))
```

The function `resid()` gives the residuals and `fitted()` gives the fitted value of height for each midparent value.

Next, we make a function to be used to bootstrap residuals in the `boot` function. This function does the following:

1. Takes input of data set `data` and vector `i`, where `i` defines the bootstrap sample. The input set `data` is of the form `datF1` above. That is, it contains the original data along with columns for the residuals and the fitted value of height for each data point.
2. Makes a new data set taking the fitted values from original data and adding bootstrapped residuals.
3. Calculate regression coefficients for the new data set.

```
heightboot.fun <- function(data, i) {
  d <- data
  d$Child <- d$fitted + d$res[i] #added residual i to each of the fitted values.
  return(coef(update(fit, data=d))[2]) #update the fit for new data set. The second coefficient is the
} #this function produces the bootstrapped estimate for the coefficients
```

Run the function to see how it works

```
heightboot.fun(datF1, sample(nrow(datF1), replace=T)) #returns fitted coefficients for data with
```

```
## Midparent  
## 1.207018
```

```
#shuffled residuals
```

This gives the slope for the fitted model for the bootstrapped data.

Next, we use this with the `boot` function:

```
(height.lm.boot <- boot(datF1, heightboot.fun, R = 10000))#call the boot function to do the bootstrapping
```

```
##  
## ORDINARY NONPARAMETRIC BOOTSTRAP  
##  
##  
## Call:  
## boot(data = datF1, statistic = heightboot.fun, R = 10000)  
##  
##  
## Bootstrap Statistics :  
##      original      bias      std. error  
## t1* 0.9312169 6.693113e-05  0.1313061
```

```
boot.ci(height.lm.boot)
```

```
## Warning in boot.ci(height.lm.boot): bootstrap variances needed for studentized  
## intervals
```

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS  
## Based on 10000 bootstrap replicates  
##  
## CALL :  
## boot.ci(boot.out = height.lm.boot)  
##  
## Intervals :  
## Level      Normal      Basic  
## 95%   ( 0.6738,  1.1885 )  ( 0.6759,  1.1917 )  
##  
## Level      Percentile      BCa  
## 95%   ( 0.6707,  1.1865 )  ( 0.6671,  1.1835 )  
## Calculations and Intervals on Original Scale
```

We see that the confidence interval for the slope does not include 0, and thus we conclude that child height does depend on midparent height. Note that we don't have any particular reason to believe that the residuals are not normally distributed. In fact, height data is typically found to be consistent with a normal. Not surprisingly then, the bootstrap CIs are very close to the theoretical CI.

Example with Bootstrapping Pairs

We will use the same data set and this time bootstrap the pairs:

```
dat=read.csv("../../data/ParentChildHeight.csv") #read in the data
datF<-dat[dat$Gender=="F",]
attach(datF)
```

```
## The following objects are masked from datF (pos = 3):
##
##      Child, Father, Gender, Midparent, Mother
```

```
fit <- lm(Child~Midparent)
```

Next, we make a function to be used in boot that calculates the slope for a bootstrapped sample:

```
#This function takes the data points defined by the vector i and calculates a new slope:
heightbootpair.fun <- function(data, i)
{   return(coef(update(fit, data=data[i,]))[2])
}
```

Now do the bootstrapping using boot:

```
(height.lm.boot.pair <- boot(datF, heightbootpair.fun, R = 10000))#call the boot function to do the boo
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = datF, statistic = heightbootpair.fun, R = 10000)
##
##
## Bootstrap Statistics :
##      original      bias    std. error
## t1* 0.9312169 0.0004115126  0.1448547
```

```
boot.ci(height.lm.boot.pair)
```

```
## Warning in boot.ci(height.lm.boot.pair): bootstrap variances needed for
## studentized intervals
```

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 10000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = height.lm.boot.pair)
##
## Intervals :
## Level      Normal              Basic
## 95%   ( 0.6469,  1.2147 )   ( 0.6521,  1.2214 )
```

```
##
## Level      Percentile      BCa
## 95%   ( 0.6410,  1.2104 )   ( 0.6294,  1.2033 )
## Calculations and Intervals on Original Scale
```

The confidence interval here is similar to that with previous example where we bootstrapped the residuals, although not identical. There is not any way to know which is superior. My guess is that the model is fairly accurate, in which case we would guess that the method of bootstrapping residuals is superior. However, we cannot be sure.

exercise #1

Use the `cats` data set in the MASS library. Conduct a regression of heart weight on body weight for females only using the `lm` function. Calculate confidence intervals for the regression parameters using the `confint` function. Then, calculate confidence intervals by bootstrapping residuals.

Permutation Tests with Linear Models

(The following is drawn heavily from the paper *Permutation Tests for Linear Models (2001)*, Anderson and Robinson, *Australian & New Zealand Journal of Statistics*)

Bootstrapping is used primarily for estimation, while permutation tests are used primarily for hypothesis testing. The bootstrap can generate confidence intervals, but it cannot generate p-values. This is because the bootstrap samples come from the original data. If the original data comes from a distribution where the null is false, then so will the bootstrap samples. There is no way to get a null hypothesis distribution from bootstrapping. In contrast, permutations generate the null hypothesis distribution by breaking true relationships in the data.

Again start with the simple linear regression model

$$y_i = \beta x_i + \epsilon_i$$

We wish to test the null hypothesis $H_0 : \beta = 0$. We can do this simply by permuting the y values relative to the x values, as described in the first set of notes on randomization tests.

Next, consider the multiple regression model:

$$y_i = \beta_1 x_i + \beta_2 z_i + \epsilon_i$$

We will assume that y, x and z are standardized to have mean 0. We will focus on testing

$$H_0 : \beta_2 = 0$$

We want test whether the partial regression of Y on Z is significant. That is, we wish to test whether there is a relationship between Y and Z while controlling for the effect of X . An appropriate test statistic is the partial correlation coefficient of Y on Z .

Now, consider the model with one predictor:

$$y_i = \beta_1 x_i + \epsilon_i$$

If the null hypothesis is true, then $\beta_1 = 0$ and the model becomes

$$y_i = \epsilon_i$$

The residuals ϵ_i are iid and therefore exchangeable. Since the response y_i equals the residual ϵ_i , then the y_i values are also exchangeable.

Now, consider the model with two predictors:

$$y_i = \beta_1 x_i + \beta_2 z_i + \epsilon_i$$

Suppose $\beta_2 = 0$. Then, we have

$$y_i = \beta_1 x_i + \epsilon_i$$

Now, the y_i 's include the effect of x_i and the residual and thus are clearly not exchangeable. That is, every z_i has an associated x_i . If we permute y_i values between z_i 's then we are permuting both the $\beta_1 x_i$ and the ϵ_i . The $\beta_1 x_i$ is not random and is thus not exchangeable.

However, suppose that we knew β_1 . Then, we could calculate

$$\epsilon_i = y_i - \beta_1 x_i$$

Now we have the true residual which is iid and therefore exchangeable. In this case, we could conduct an exact permutation procedure. The idea is that we want to look at the effect of Z independent of the effect of X . and thus we want to permute the effect of X along with Y .

Let's do an example with this. We will first generate data:

```
set.seed(1223)
n<-30
xdat<-runif(n,0,10) #generate x values
zdat<-runif(n,0,10) # generate z values
beta1<-0.25 #true regression coefficient for y on x
beta2<-0.1 #true regression coefficient for y on z
sd0=1 #standard deviation of residuals

ydat<-beta1*xdat+beta2*zdat+rnorm(n,sd=sd0) #generate y values that depend on x but not z.
```

Let's first analyze using the normal approximation. (which is, in fact, correct because we generated the residuals as iid normal)

```
yxzmod<-lm(ydat~xdat+zdat)
summary(yxzmod)
```

```
##
## Call:
## lm(formula = ydat ~ xdat + zdat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.7345 -0.3067  0.1257  0.6496  1.4311
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.52491    0.41488   1.265   0.217
## xdat          0.14515    0.06113   2.374   0.025 *
## zdat          0.07539    0.05420   1.391   0.176
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8876 on 27 degrees of freedom
## Multiple R-squared:  0.2291, Adjusted R-squared:  0.172
## F-statistic: 4.013 on 2 and 27 DF,  p-value: 0.02979
```

Now, run the proposed permutation procedure. In this case, I know the true regression coefficient $\beta_1 = 0.25$. Therefore, I can use it in my permutations. We will use the `pcor` function from `rsq` library to calculate partial correlation coefficient. This will be the test statistic.

```
library(rsq)
nrep=1000 #number of permutation replicates
```



```

#Calculate true residuals:
rxy<-ydat-beta1*xdat

permout<-vector(length=nrep)

#Now, do the permutations
for(r in 1:nrep)
{
  ynew<-beta1*xdat+sample(rxy) #make permuted y values using the true model for x
  pmod<-lm(ynew~xdat+zdat)
  permout[r]<-pcor(pmod)$partial.cor[2]
}

sum(abs(permout)>pcor(yxzmod)$partial.cor[2])/nrep

## [1] 0.156

```

We see that the p-value is quite close to the normal approximation value.

The Reduced Model

The problem, of course, is that we don't know β_1 . A number of methods have been proposed for solving this problem.

This method was proposed by Freeman and Lane (1983):

- 1) Estimate β_1 using the X,Y data alone.
- 2) Estimate residuals using the fitted x values: $\hat{\epsilon} = y_i - \hat{\beta}_1 x$
- 3) In each permutation replicate, construct new y-values from the fitted model and the permuted residuals:

$$y_{perm} = \hat{\beta}x + \epsilon_{perm}$$

Here is an example. This is very similar to my previous example, but now I need to estimate **beta1**.

```

set.seed(1223)
n<-30
xdat<-runif(n,0,10) #generate x values
zdat<-runif(n,0,10) # generate z values
beta1<-0.25 #true regression coefficient for y on x
beta2<-0.1 #true regression coefficient for y on z
sd0=1 #standard deviation of residuals

ydat<-beta1*xdat+beta2*zdat+rnorm(n,sd=sd0) #generate y values that depend on x but not z.

```

Here again is the analysis using the normal approximation.

```

yxzmod<-lm(ydat~xdat+zdat)
summary(yxzmod)

```

```
##
## Call:
## lm(formula = ydat ~ xdat + zdat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.7345 -0.3067  0.1257  0.6496  1.4311
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.52491    0.41488   1.265   0.217
## xdat          0.14515    0.06113   2.374   0.025 *
## zdat          0.07539    0.05420   1.391   0.176
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8876 on 27 degrees of freedom
## Multiple R-squared:  0.2291, Adjusted R-squared:  0.172
## F-statistic: 4.013 on 2 and 27 DF,  p-value: 0.02979
```

Next, let's do it by permutations.

```
library(rsq)
nrep=1000 #number of permutation replicates

#First, fit the regression of y on x alone:
xymod<-lm(ydat~xdat)
#Next, get the fitted values and residuals from the x-y regression
fxy=fitted(xymod)
rxv=residuals(xymod)
b1hat=coef(xymod)[2]
permout<-vector(length=nrep)

#Now, do the permutations
for(r in 1:nrep)
{
  ynew<-fxy+sample(rxv) #make permuted y values
  pmod<-lm(ynew~xdat+zdat)
  permout[r]<-pcor(pmod)$partial.cor[2]
}

sum(abs(permout)>pcor(yxzmod)$partial.cor[2])/nrep
```

```
## [1] 0.159
```

The p-value is again very close to the one from the normal distribution, but less so than for the previous method. This is likely because we are estimating β_1 .

Permute the Y values

Manley(1997) proposed that we simply permute the y-values and ignore the issues that we mentioned above.

```
library(rsq)
nrep=1000
permout<-vector(length=nrep)
for(r in 1:nrep)
{
  pmod<-lm(sample(ydat)~xdat+zdat)
  permout[r]<-pcor(pmod)$partial.cor[2]
}

sum(abs(permout)>pcor(yxzmod)$partial.cor[2])/nrep

## [1] 0.294
```

The p-value appears to be less accurate than for the previous method.

Permuting Residuals from the Full Model

Ter Braak (1992) proposed permuting the residuals of the full model. That is, we calculate residuals

$$\hat{\epsilon}_i = y_i - \hat{\beta}_1 x - \hat{\beta}_2 z$$

The difference from the reduced model is that here we subtract out the effect of z as well.

```
library(rsq)
nrep=1000
permout<-vector(length=nrep)

yxzmod<-lm(ydat~xdat+zdat)
ryxz<-residuals(yxzmod)
fyxz<-fitted(yxzmod)

for(r in 1:nrep)
{
  ynew<-fyxz+sample(ryxz) #make permuted y values
  pmod<-lm(ynew~xdat+zdat)
  permout[r]<-pcor(pmod)$partial.cor[2]
}

sum(abs(permout)>pcor(yxzmod)$partial.cor[2])/nrep

## [1] 0.53
```

This p-value also seems less accurate than the first method.

Comparing the Methods

Anderson and Robinson (2001) compare these methods and show that they all converge asymptotically to the same thing. That is, all work as approximate tests for a partial regression coefficient, provided that the predictors are independent. However, their performance at reasonable sample sizes is different. Furthermore, the issue of independence is important. The question of whether you separate the influence of $\mathbf{X}$ and $\mathbf{Z}$ in the permutations makes a difference with dependent data.

You will explore this in the next homework assignment.